

A Practical Guide to Convolutional neural network

TOPIC -- CONVOLUTIONAL NEURAL NETWORK

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

-- 01 --

Natural language processing CNNs have also been explored for natural language processing. CNN models are effective for various NLP problems and achieved excellent results in semantic parsing, search query retrieval, sentence modeling, classification, prediction and other traditional NLP tasks. Compared to traditional language processing methods such as recurrent neural networks, CNNs can represent different contextual realities of language that do not rely on a series-sequence assumption, while RNNs are better suitable when classical time series modeling is required.

-- 02 --

Max norm constraints Another form of regularization is to enforce an absolute upper bound on the magnitude of the weight vector for every neuron and use projected gradient descent to enforce the constraint. In practice, this corresponds to performing the parameter update as normal, and then enforcing the constraint by clamping the weight vector $w \rightarrow \{\vec{w}\}$ of every neuron to satisfy $\|w\|_2 < c$. Typical values of c are order of 3–4. Some papers report improvements when using this form of regularization.

-- 03 --

Fully connected layer After several convolutional and max pooling layers, the final classification is done via fully connected layers. Neurons in a fully connected layer have connections to all activations in the previous layer, as seen in regular (non-convolutional) artificial neural networks. Their activations can thus be computed as an affine transformation, with matrix multiplication followed by a bias offset (vector addition of a learned or fixed bias term).

-- 04 --

Hierarchical coordinate frames Pooling loses the precise spatial relationships between high-level parts (such as nose and mouth in a face image). These relationships are needed for identity recognition. Overlapping the pools so that each feature occurs in multiple pools, helps retain the information. Translation alone cannot extrapolate the understanding of geometric relationships to a radically new viewpoint, such as a different orientation or scale. On the other hand, people are very good at extrapolating; after seeing a new shape once they can recognize it from a different viewpoint. An earlier common way to deal with this problem is to train the network on transformed data in different orientations, scales, lighting, etc. so that the network can cope with these variations. This is computationally intensive for large data-sets. The alternative is to use a hierarchy of coordinate frames and use a group of neurons to represent a conjunction of the shape of the feature and its pose relative to the retina. The pose relative to the retina is the relationship between the coordinate frame of the retina and the intrinsic features' coordinate frame. Thus, one way to represent something is to embed the coordinate frame within it. This allows large features to be recognized by using the consistency of the poses of their parts (e.g. nose and mouth poses make a consistent prediction of the pose of the whole face). This approach ensures that the higher-level entity (e.g. face) is present when the lower-level (e.g. nose and mouth) agree on its prediction of the pose. The vectors of neuronal activity that represent pose ("pose vectors") allow spatial transformations modeled as linear operations that make it easier for the network to learn the hierarchy of visual entities and generalize across viewpoints. This is similar to the way the human visual system imposes coordinate frames in order to represent shapes.

-- 05 --

The "loss layer", or "loss function", exemplifies how training penalizes the deviation between the predicted output of the network, and the true data labels (during supervised learning). Various loss functions can be used, depending on the specific task. The Softmax loss function is used for predicting a single class of K mutually exclusive classes. Sigmoid cross-entropy loss is used for predicting K independent probability values in $[0, 1]$. Euclidean loss is used for regressing to real-valued labels $(-\infty, \infty)$.

-- 06 --

Although fully connected feedforward neural networks can be used to learn features and classify data, this architecture is generally impractical for larger inputs (e.g., high-resolution images), which would require massive numbers of

neurons because each pixel is a relevant input feature. A fully connected layer for an image of size 100×100 has 10,000 weights for each neuron in the second layer. Convolution reduces the number of free parameters, allowing the network to be deeper. For example, using a 5×5 tiling region, each with the same shared weights, requires only 25 neurons. Using shared weights means there are many fewer parameters, which helps avoid the vanishing gradients and exploding gradients problems seen during backpropagation in earlier neural networks. To speed processing, standard convolutional layers can be replaced by depthwise separable convolutional layers, which are based on a depthwise convolution followed by a pointwise convolution. The depthwise convolution is a spatial convolution applied independently over each channel of the input tensor, while the pointwise convolution is a standard convolution restricted to the use of 1×1 kernels.

-- 07 --

Convolutional layers In a CNN, the input is a tensor with shape: (number of inputs) $\times$ (input height) $\times$ (input width) $\times$ (input channels). After passing through a convolutional layer, the image becomes abstracted to a feature map, also called an activation map, with shape: (number of inputs) $\times$ (feature map height) $\times$ (feature map width) $\times$ (feature map channels). Convolutional layers convolve the input and pass its result to the next layer. This is similar to the response of a neuron in the visual cortex to a specific stimulus. Each convolutional neuron processes data only for its receptive field.

-- 08 --

Dilation Dilation involves ignoring pixels within a kernel. This reduces processing memory potentially without significant signal loss. A dilation of 2 on a 3×3 kernel expands the kernel to 5×5 , while still processing 9 (evenly spaced) pixels. Specifically, the processed pixels after the dilation are the cells (1,1), (1,3), (1,5), (3,1), (3,3), (3,5), (5,1), (5,3), (5,5), where (i,j) denotes the cell of the i-th row and j-th column in the expanded 5×5 kernel. Accordingly, dilation of 4 expands the kernel to 7×7 .